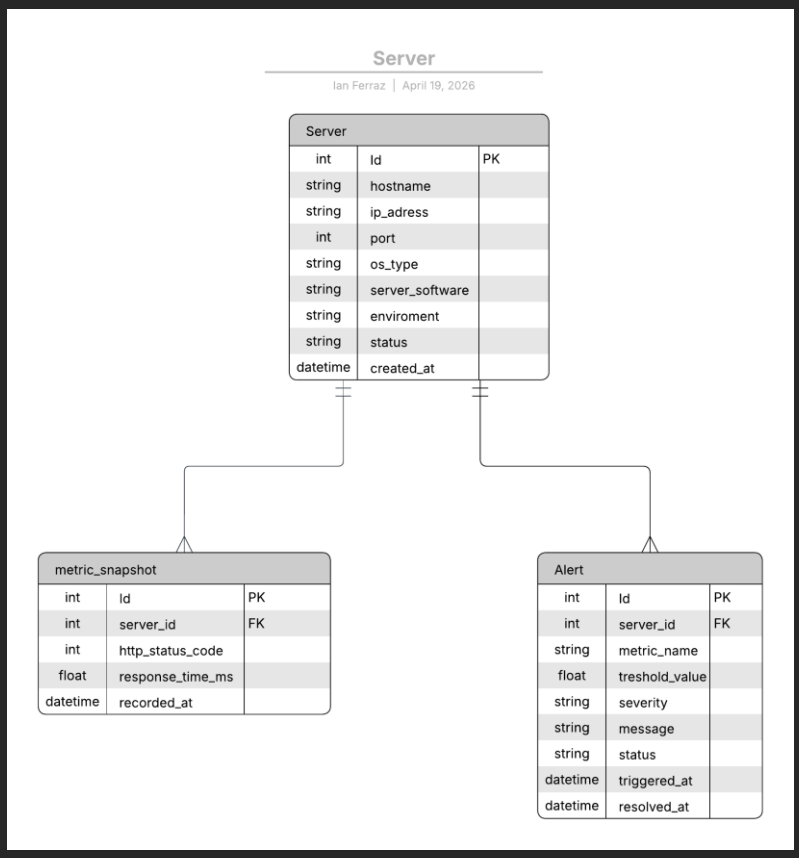
AI usage note:

Questions and sub-tasks in which AI assistance was used are identified by a "*" at the question number or task header identifier, followed by AI usage note detailing each use case at the end of every answer.

Candidate: Ian Matsuhara Ferraz

Part1:



Q1:

A)

For the database model, I chose to utilize three seperate tables, server, metric_snapshot and alert, rather than keeping everything in only one table since using a single db table would be an issue for static data such as hostname, ip_address and os_type. These data points are never changing between cycles, and placing everything in one same table makes these fields always repeat on every row of the monitoring history log, cluttering the log since having them there is basically useless and does not assist in monitoring observability. Furthermore, a change to the ip_address field for example would in turn require updating every historical record rather than just a single row. Another negative impact of putting everything in one single table would be that whenever running queries about the current state of the server, these would unecessairly scan monitoring history log. Seperating into three distinct tables eliminates all these problems, making each table responsible for a specific concern, server answers only what a server is, metric_snapshot answers only what was collected overtime and Alert answer only what went wrong and when.

The monitoring process thought of when designing this diagram was to implement monitoring as a cron job, scheduled to run automatically at fixed intervals, it makes sense to choose a cron since crons give us periodic snapshots rather than just continuous execution as in running a daemon. The cron runs, collects, writes to the database and exits, this keeps the process stateless, since each execution is fully independent from the previous one, and simple to maintain. Moreover, the relationship between tables is represented in

the diagram using crow's foot notation, since both metric_snapshot and alert have a many to one relationship with server. In other words, one server can accumulate many snapshots and alerts overtime or none at all if it was just registered for example, but each alert and each snapshot must always belong to only ones server, and cannot exist without a server, represented by the double lines on the server side on each line connecting the tables to server, this is also enforced by the server_id foreign key in both metric_snapshot and alert tables.

The server table holds only static information about each registered server, with the exception of the status field, which is an enum and the only field of the table updated dynamically by the monitoring script, it contains only the values active, inactive or maintenance, which except for maintenance are updated automatically by the monitoring script on each cycle based on the HTTP response received. A 200 set server status to active, a failure or timeout sets it to inactive and maintenance is special since its not changed by the script, its reserved for manual override via the frontend whenever the server is taken offline for maintenance reasons.

The metric_snapshot table records and logs the result of each HTTP check performed by the cron job that's why server_id is a foregin key referencing server.id, so that each snapshot row is linked to exactly one server record, allowing to query all snapshots for a specific server without duplicating the server's static data on every row, making it so that http_status_code, response_time_ms and recorded_at metrics can be easily retrieved from when that cron job ran for that specific server. Each row in this table represents one monitoring cycle for one server, making this table a natural audit log of the cron job's execution history.

The Alert table is populated by the monitoring script in the same cycle as the metric_snapshot table whenver a collected metric falls outside the expected treshholds. I chose to include a resolved_at field at this table as a seperate nullable field rather than just a boolean since it allows the frontend to display both when the alert was triggered and when it was resolved, this is most useful when concerning operational visibility rather than just a simple open/closed flag.

At last, the complete monitoring flow that this diagram supports would be: A cron job runs at fixed intervals, makes na HTTP request to each registered server, writes the result to metric_snapshot, compares the response against defined tresholds, if any treshold was excceeded, it writes to alert and updates the status field on server accordaingly. This would all be accomplished within a single process and without any external dependencies, the fact that all three tables are written in the same cycle by the same process makes this schema functional, each table playing a distinct role wihtin a single unified flow.

***B)**

AI usage note:

The solution here applied is structured over a single index.php file containing all layers of the application, following a specific order, php backend logic at the top, data fetch queries in the middle, and plain HTML, CSS and javascript at the bottom. PHP is ther server sided language handling all data operations, and mysqli the built in php extension used to connect and communicate with the MySQL database.I chose to use this single file formart given the context of this assessment, since no build step, no framework, and no routing configuration were required in this implementation.

The gui for this application was conceptualized as a dashboard with a top navigation bar, a list of servers as the main content shown as table containing each server's hostname, ip_address, port, status and most recent http_status_code and response_time_ms pulled from the metric_snapshot table. Each row on the table also has edit, maintenance and delete action buttons, this accounts for the writing operations request of the assignment. Adding and editing operations here are handled through pop ups so that the user never needs to leave the dashboard page.

Every pop up form submission includes a hidden field identifying which operation to perform, the backend in turn reads this field and routes the request to the correct function without needing separate pages for each operation type(register, edit, maintenance or delete). In turn, each function handles its database interaction in a way that prevents user input from being interpreted as part of the query, which is good practice since its standard protection against sql injections. After an operation is completed, the page redirects back to itself, effectively being updated, this also prevents the browser from accidentally resubmitting the same form if the user manually refreshes the page. The register operation is the exception to this behavior since after processing its required to return the ID of the newly created server.

The server list query is written specifically to return exactly one row per server, pulling only the most recent monitoring snapshot for each one, this prevents scenarios such as when a server with multiple snapshots in history would appear multiple times in the table, being redundant and not useful for monitoring. Delete operations also clean up all monitoring and alert records linked to a server before removing it, rather than just relying on the database to do the work automatically, this is better since automatic database compensation is not guaranteed depending on how the database was configured, and since here there's no say on how the database would have been configured, implementing this failsafe is beneficial.

At last, there were two notable adaptions Kiro made by its own and are worth mentioning. The first one would be in prompt 3, Kiro caught that the database connection was being closed before the server list query ran, which if left untreated would lead the table to load empty with no visible error. Kiro fixed this by moving the database connection close after the query. In prompt 4, Kiro improved the edit form so that clicking on the edit button on any row would instantly open a pre-filled pop up using data already present on the page, instead of needing to reload the page to fetch that server's data from the database again. Both adaption were reviewed and accepted.

** It's also worth noting that the single file format approach I chose was motivated by simplicity's sake given the context of this assessment, however, in a production environment, php logic css and javascript would need to be separated into distinct files, each concerning its own layer of the application, and the solution would also need to include other aspects such as authentication, protection and error handling.

Link to github repository:
https://github.com/YoshuJumpMaster/technial-assessment/tree/main/q1

Prompt 1:

"
Create a single index.php file for a server monitoring dashboard. The backend should use PHP with MySQLi to connect to a MySQL database. It should handle four operations via POST requests: registering a new server (insert into Server table), editing an existing server's static fields (hostname, ip_address, port, os_type, server_software, environment), manually setting a server's status to maintenance, and deleting a server. Each operation should be triggered by a hidden action field in the form submission. The database has three tables: Server (id, hostname, ip_address, port, os_type, server_software, environment, status, created_at), metric_snapshot (id, server_id, http_status_code, response_time_ms, recorded_at) and Alert (id, server_id, metric_name, threshold_value, severity, message, status, triggered_at, resolved_at). Write only the PHP backend logic for now, no HTML or CSS yet. Add comments separating each operation clearly.
"

Explanation:

This prompt was scoping the backend only before actually building any interface around it. Each operation in the bakend is isolated in its own function via the hidden action field, keeping all logic in one place without requiring a router. Accepted, but with adjustment in the following prompt.

Prompt 2:

“
In the current index.php, all four operations return JSON responses after success. Since this is a single file PHP application that renders HTML, replace these responses with a redirect back to index.php. Keep the JSON error responses for failure cases since those are still useful for debugging. Also keep the JSON success response only for registerServer since it returns the new server's insert_id which may be useful later.
”

Explanation:

This was a necessary adjustment before adding HTML, all success responses were returning raw json which the browser would display instead of the dashboard. Kiro applied the required edits and left registerserver() unchanged as requested. Accepted without modifications.

Prompt 3:

“
Now add the HTML structure to the same index.php file. The page should have a server list table displaying all servers with columns for hostname, ip_address, port, status, last http_status_code and last response_time_ms, the last two being pulled from the most recent metric_snapshot for each server ensuring only one snapshot row per server is returned. Each row should have edit, set maintenance and delete action buttons. Above the table there should be an Add Server button that opens a form to register a new server. The edit action should populate a form with the server's current values. Keep the PHP backend block at the top of the file and the HTML below it, separated by a clear comment.
”

Explanation:

In this prompt I prompted kiro to design the html structure, detailing the concept I had envisioned and the constraints. The correlated subquery was also specified explicitly to avoid a direct join returning multiple rows per server, as this would break the dashboard table. Kiro also caught the connection closing before the query ran and fixed it in this step. Accepted without modifications.

Prompt 4:

“
Now add CSS styles and JavaScript to the same index.php file. The CSS should style the page as a clean, minimal dashboard: a top navigation bar with the page title, a styled HTML table with alternating row colors, status badges with colors reflecting the server status (green for active, red for inactive, yellow for maintenance), and a pop up for the add and edit forms. The JavaScript should handle opening and closing the pop up. Add the CSS inside a style tag in the head and the JavaScript inside a script tag before the closing body tag, both clearly commented.
”

Explanation:

In this prompt I prompted kiro to design the CSS for the page, giving it description of what I had conceptualized, a minimal dashboard, and the pop up design. For the styling, I also decided to go with status badges with semantic colors, as they would be the most readable indicator of a server’s health at a glance. In this step Kiro also refactored the edit form to use data that was already in the page instead of reloading without being prompted to do so. Kiro also made pop-ups closeable via backdrop click and escape key without being prompted. Accepted without modifications.

Assumption summary:

For this solution, I assumed that the database I had proposed and its three tables already existed and that they were already accessible with the database credentials configured at the top of index.php. No schema creation or migration logic was included. The delete operation also assumes that a automatic deletion of linked records by the database may not be configured and handles dependent row cleanup manually. There’s also an assumption concerning the monitoring data displayed in the table, this implementation assumes the data in metric_snapshot to be written by separate processes, as proposed in q1c, meaning the dashboard depends on that process to have data to display in the monitoring columns.

C)

The primordial step in the monitoring process would be to retrieve the list of servers to be monitored by querying the server table and filtering out any servers that with status set to maintenance, since by design those would be intentionally offline, hence checking them would be pointless. After running this query, the expected output of this query would be the list of servers were actually interested on, and would hence be what well use for the rest of this cycle.

Once we have the list, we can proceed to check whether each serving is up and running by making an http request to it and recording the response. Were using http requests since they capture two relevant metrics, http status code and the time it took for the server to respond in milliseconds. Following the process, both of these metrics would be written alongside the timestamp for when the check ran in a new row inside the metric_snapshot table, being linked to their respective server via server_id. Based on this information, the server status field on the server table is also updated, a 200 returned http status code would set status to active while a failure or timeout would set it to inactive. The maintenance field is being purposefully excluded in this process since this process here being described would be designed to be automatic, and the maintenance field, per previous definitions is designed to be reserved for manual intervention via the front end of the application, hence it should not be overwritten by a monitoring script.

Following the process, we would proceed to connect to each server via ssh and run commands remotely, similarly to what a system administrator would do manually to check a server from terminal. Commands would be used to validate metrics such as whether the web server process is currently running on the machine, if the process is not found the script would automatically proceed to automatically write a line to the alert table with the server_id alongside a timestamp, indicating what was detected from the command outputs. Logically the resolved_at field would be left null in this case until the issue is resolved and the field status of the field would then be manually changed to solved.

Following the process, we would utilize commands to extract current cpu and memory usage, if either of these metrics exceeds a defined threshold, the script also writes to the alert table, like in the previous step, and of course they would have their separate thresholds being separate values and separate checks, and so they also write alerts separately. This is more intuitive since it assists in distinguishing which specific metric triggered an alert when going through the dashboard.

Having the steps of the process itself defined, the execution model I chose was a cron job rather than a persistent daemon. I decided to go with a cron because it runs at fixed intervals, collects the data, writes to the database and exits, making each of its executions completely independent from one another. This is beneficial because it keeps the process stateless and simple to maintain since there’s no long running process to monitor, no memory consumption by the process to manage, and if a single cycle fails for any

reason it doesn't affect the next one. Crons also have the added benefit of being able to be scheduled during low system usage time windows, such as in early mornings, were system activity is expected to be lower, even more beneficial when concerning the memory consumption. In contrast, a daemon running continuously would only add operational complexity without any meaningful benefit given this scenario, of q periodic monitoring necessity.

Before the checks already specified, there are other metrics that would also be relevant to monitor in a real infrastructure context. One of the most critical omissions in this case would be disk usage, a server can be up and responding to http requests while simultaneously running out of disk space, which would eventually lead to failures that the checks here described wouldnt detect. Another metric also useful to verify would be open port scanning, since it offers insight as to if the expected services are listening on the correct ports. SSL certificate expiry date is another useful check as well, since an expired certificate would lead the server to become unreachable for clients without any change to the server uptime status itself.

Q2)

First of all, since we're dealing with a critical system package update, the first step would be to backup the system's state through snapshot, copy the database to a backup DB, and document the current package version being used and make it recoverable, for future rollback in case of any break during the update. Its also useful to consult the package update changelog and release notes to get a complete picture of what's in fact being changed. This way, we have a narrower scope into possible issues and breaking points during the update, making it easier to devise more targeted and precise remedial actions in case of errors.(https://wiki.archlinux.org/title/System_maintenance , https://wiki.archlinux.org/title/System_backup , https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/back-up-and-restore-of-sql-server-databases?view=sql-server-ver17)

Before actually performing the update on production itself we should also simulate it on a staging environment that mirrors production, this way eventual bugs can be tackled beforehand with planned action and contingencies. Simulating the upgrade on a test environment also allows us to run post update test-suites thoroughly to ensure no bugs that weren't noticed are left unhandled. Aside from running post upgrade test suites, its also crucial for us to run rollback testing as well, since the rollback scenario must also be thoroughly planned, written and tested before actually being performed on prod, just like the update itself. Only after update and rollback pass successfully through testing should we move forward. (https://continuousdelivery.com/principles/ , https://horizondatasys.com/computer-feature-update-rollback/)

As for the downtime aspect, I believe the best approach would be to run both current and post-update versions in parallel and gradually redirect traffic from the current to the new version, while keeping close attention to logs for monitoring error and performance indicators during this process. In case of unexpected behavior such as unusually high errors rates and high latency, redirect traffic back to old stable system version. Given that this process in itself has a chance of surging effects on the client's side, its also useful for us to notify clients beforehand of an upcoming update as well as the planned update window. This is useful for even if after all testing there is no expectation for downtime, a planned update window places both stakeholders and internal resources in alignment with system behavior during that time.(https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/introduction.html , https://www.ibm.com/think/topics/blue-green-deployment)

Lastly, even if everything works as expected and the update appears to have been successful, its pivotal to keep monitoring system behavior indicators such as logs, error rates, and performance metrics throughout system operation for some time to consider the update stable and fully complete. This time window can very from scenario to scenario, but given that here we're dealing with a critical system, I would suggest 3 days, I believe that's sufficient time to observe system behavior in production and close the update successfully.

As for challenges, given that this is a critical system, some of the potential challenges I could picture given a package update would include, for starters, dependency conflicts. The old package may be running on libraries that have not been updated to support the new package, so that's something important to check, and its also important to see what reflex updating those libraries in turn would have on the overall system. Another challenge may come out as insufficient test coverage, if testing is insufficient, the update process will proceed and bugs will eventually show up in prod, hence why thorough testing during updates is crucial.(https://en.wikipedia.org/wiki/Dependency_hell)

Q3)

Given that this would be a Command Line Interface application for internal use, the first thing that strikes me as inspiration would be to use the CLI application I currently use on my day to day, like git, pacman or yay from arch linux, which I use greatly on my personal projects, as inspiration for this answer. Since the focus here is on the deployment process given hundreds of concurrent simultaneous users across multiple teams, the challenge is ensuring that all these people on different machines can install and use the application reliably and receive updates without disruption. As such, the first thing that needs to be done would be to package the application properly using using

setuptools, which is python's standard build tool for packaging applications, as described in Python's packaging user guide (https://setuptools.pypa.io/en/latest/index.html , https://setuptools.pypa.io/en/latest/setuptools.html, https://packaging.python.org/en/latest/overview/ , https://packaging.python.org/en/latest/tutorials/packaging-projects/). This would involve creating a pyproject.toml configuration file defining the application's name, version, dependencies and most importantly the terminal inputs users will use to run it. Once packaged this way, the application can be seamlessly installed using pip install command, the same way any python library would be installed, following the standard installation process documented in pip's official documentation (https://packaging.python.org/en/latest/guides/tool-recommendations/, https://pip.pypa.io/en/stable/ , https://pip.pypa.io/en/stable/getting-started/).

Since this is an internal company application, publishing it to Python's public package registry isn't an option, so we would need to setup a private internal registry to account for internal distribution, an option for this would be using tools such as pypiserver, a lightweight private package registry, giving company full control over versioning access and updates. (https://packaging.python.org/en/latest/tutorials/packaging-projects/ , https://pypi.org/ , https://packaging.python.org/en/latest/tutorials/installing-packages/) Given the internal registry, the command users would need to run to install the application would also need to point to that internal registry, so it would look something like:

```
pip install --index-url https://internal-registry....  application_name
```

Every release of the application would follow semantic versioning, the major.minor.patch convention where each version number already communicates the nature of that change, this makes things more intuitive and informative. For example, a patch like 1.0.1 would indicate a bug fix, 1.1.0 would indicate a minor new feature add for example, 2.0.0 on the other hand would indicate major changes, prompting greater attention when updating to make sure nothing in existing usage breaks. Semantic versioning is common practice in software development due to its practicality and clear communication of intent. A version number alone tells the user whether they need to be cautious about updating or not, which is especially important when hundreds of people are using the same app, like in this scenario, so all in all, this versioning schema makes users more conscious when updating(https://semver.org/, https://www.geeksforgeeks.org/software-engineering/introduction-semantic-versioning/). A nice feature on the application to have in the sense of updates and versioning would be a feature where the CLI itself checks the registry on startup and prints a message on terminal in case a newer version is available, accompanied by the version number, only to be informative, not forcing any updates.

For preventing major updates that could potentially break current usage or existing workflows, I believe a phased rollout would be the best solution, releasing first to a small pilot group, collecting feedback, and only then pushing to the rest of the users. It would also be important for the app to be backwards compatible for at least one major version cycle, this ensures that teams have time to adapt without being forced to update immediately.

Seeking efficiency, I would also automate the release process for this application, a CI/CD pipeline would be most adequate, since it automatically tests and publishes whenever code is merged to the tools repository. Having tests run automatically whenever developers merge changes, followed by automatic validation and publishing to the registry without any manual intervention greatly decreases human error and standardizes release process. This approach follows continuous delivery principles, which establish that software should always be in a releasable state and that release processes should be fully automated to minimize human error(https://continuousdelivery.com/principles/, https://unidel.edu.ng/focelibrary/books/Continuous%20Delivery%20Reliable%20Software%20Releases%20through%20Build,%20Test,%20and%20Deployment%20Automation%20by%20Jez%20Humble,%20David%20Farley%20(z-lib.org).pdf , https://www.geeksforgeeks.org/devops/what-is-ci-cd/ , https://codefresh.io/learn/ci-cd/ , https://www.redhat.com/en/topics/devops/what-is-ci-cd)

Moreover, thinking more about how this application would be used day to day, since different teams may have different Python versions or conflicting packages installed on their machines, it would be best to standardize usage of this CLI through virtual environment on the user's machine, using an isolated environment makes sense since its an isolated python installation that doesn't interfere with anything else on the user's machine. Seeking efficiency and a seamless process for the user, a single installation script could be used for handling environment creation and installation. This is considered best practice in Python development because it guarantees that the application always runs with the required dependencies that it was built for, regardless of what else is installed on the user's machine, as recommended in the official python documentation. (https://docs.python.org/3/library/venv.html , https://stackoverflow.com/questions/72835581/explain-why-python-virtual-environments-are-better , https://medium.com/data-science/why-you-should-use-a-virtual-environment-for-every-python-project-c17dab3b0fd0)

Since hundreds of people would be using the tool simultaneously, the cli itself should also be stateless, each execution should be completely independent, guaranteeing that if the app calls a backend, an API or a database, that layer will in turn be responsible for handling concurrency. Keeping cli stateless is well established in distributed systems design, given that by their nature, when each execution is independent, the application scales to accommodate the number of users without requiring any coordination between instances. This decision also aligns the architecture of the project with the

stateless component principle. (https://12factor.net/ , https://en.wikipedia.org/wiki/Twelve-Factor_App_methodology , https://www.geeksforgeeks.org/blogs/what-is-twelve-factor-app/ , https://www.geeksforgeeks.org/system-design/stateless-and-stateful-systems-in-system-design/ , https://www.freecodecamp.org/news/stateful-vs-stateless-architectures-explained/)

At last, none of the aforementioned steps would be sufficient if not accompanied by good documentation. For that reason, it would be interesting that every command in the application had a built in –help flag, moreover, internal company knowledgebase should include a wiki with installation and app usage instructions, ensuring smooth introduction to new team members requiring minimal senior support.

Q4)

In my current position I work at the professional services team, being responsible for ensuring clients successfully implement, customize and extract value from the company's products. This translates into day-to-day activities varying from developing new features into the system to managing sub-routines that run daily in the client's system such as import scripts. Perhaps the work I perform in the latter is more relevant in answering this question, so I'll use it to illustrate my answer.

After dealing with import scripts in a day to day basis, I eventually developed a level of repetition in my work, aside from specific cases which may distinguish a client's implementation from another, such as custom script logic or integration, there are tasks that often strike me as something unnecessary for a developer to be spending hours on. As such, I am constantly aware of such repetition in tasks and have brought up initiatives to automate such routines through the use of AI aiming to improve workflow efficiency.

More specifically to the question's ask, it doesn't take me long after performing a task repeatedly before starting to reflect and analyze on the potential ROI of automations. If I was to put a precise number of times to that answer, I would say doing the same task three times already sparks in me the interest in seeking more efficient workflows, performing it 5 or more times already prompts me into action for researching and analyzing the validity of automating that task.

As for the reasoning behind these numbers, I believe that performing a task once is for learning, twice is still learning but more in the sense of starting to get the hang of it and understanding specifics such as edge cases, recognizing patterns, limitations and expected vs unexpected behaviors. By the third time onward, I believe learning and experience gathering gradually start to go down as the costs of manually performing something where that time could be better allocated starts to go up.

Having all that said, I believe its also important to ponder that not all tasks that CAN be automated SHOULD be automated. When executing the same task three or more times the thought process I often go through is to reflect on the relative stability of that task, meaning how much of that task is consistent throughout multiple iterations and how much of it can change over iterations, how frequent are those changes? I think its important to weigh these aspects because if the frequency of changes outweighs the overall consistency throughout iterations, such an automation would require too much maintenance and modifications overtime, defeating its purpose. Aside from that, I also ponder other aspects such as what would the ROI actually be, would it justify the effort of developing that automation? How critical is the task at hand? And how impactful it is truly? Are there sufficient instances of human errors to justify automation? These questions are an illustration of the thought processes I go through daily when observing tasks and contemplating automation. I believe its important to have a thorough and detailed analysis of the situation before actually implementing anything. Afterall, even if something is repetitive, if its overall impact in productivity is dismissible perhaps automation isn't justified after all.

Part2:

***Q5)**

A)

AI usage note:

I decided here to go with a configurable license plate validator. Given a config file with plate string and constraints, the module will check if constraints are satisfied. Since validation constraints are in the config itself, the module should handle any plate format without any code change, so no validation criteria should be hardcoded. The implementation decision made was to separate config loading from validation logic into two separate functions, this way, each function has a single, well-defined responsibility and can also be tested independently. I also decided to contemplate the two types of plates currently in circulation in Brazil as test cases, old plates and mercosul standard plates, I made this to make the solution more grounded in

reality and better illustrate its applicability. The solution was developed iteratively using Kiro as an AI assistant, with each step reviewed and validated before proceeding to the next. Prompts and workflow process logic are documented below, including reasoning behind each decision and what was accepted, adjusted or notable in each output.

Link to GitHub repository:
https://github.com/YoshuJumpMaster/technial-assessment/tree/main/q5

prompt 1:
“
I need to implement a Python module responsible for validating license plates. The module will receive two inputs: the name of a .cnf JSON config file (located in the same directory as the module) and the license plate string to validate. The JSON file may contain up to three attributes: length (total string length), letters (number of letter characters), and numbers (number of digit characters). Not all attributes are guaranteed to be present,so the validation should only enforce the constraints that are actually defined in the file. Write only the module code for now, no tests yet. Make sure the config loading is isolated in its own function, separate from the validation logic itself. Add docstrings explaining the purpose of each function.
”
Explanation:

The first prompt is intended to set the structure for the module only, not setting up any configs or tests yet. The separation between load_config and validate_plate was deliberate for clearly separating both functions with distinct responsibilities and distinct failure handling. Having the separation of functions clearly defined makes each function indepedently testable, which was the immediate practical benefit. Kiro proceeded to develop the module exactly as specified. Accepted without modifications.

prompt 2:
“
Now create two .cnf JSON config files to be used for testing the module, representing the two Brazilian license plate formats currently in circulation. The first, brasil_old.cnf, should represent the following format: 7 characters total, 3 letters and 4 numbers (e.g. ABC1234). The second, brasil_mercosul.cnf, should represent the current Mercosul format, also 7 characters total but with 4 letters and 3 numbers (e.g. ABC1D23). Both files should be flat JSON objects using only the

length, letters, and numbers keys as defined in the load_config function in license_plate_validator.py
”

Explanation:

Rather than creating a single generic config, I decided to be more reflective of reality of vehicles currently in circulation in Brazil and chose to represent both plate formats, making the test more meaningful. A plate valid under one format should correctly fail under the other. Both files were verified by inspecting their contents before proceeding.

prompt 3:
“
Now write the test suite for the license plate validator module using Python's built-in unittest framework. The tests should cover: a valid plate that satisfies all constraints in brasil_old.cnf, a valid plate that satisfies all constraints in brasil_mercosul.cnf, a plate that fails the length constraint, a plate that fails the letters constraint, a plate that fails the numbers constraint, a plate that fails multiple constraints simultaneously, and a case where the config file does not exist. Each test should include a brief comment explaining what specific behavior it is asserting. Do not use any mocking libraries, the tests should load actual .cnf files from disk.
”

Explanation:

For testing, I decided to exclude mocking libraries since config loading from disk is part of the modules intended behavior, hence its not an external side effect to be isolated. Missing config file case was also included even though the assignment states that files will be there in production but doesnt explicitly say always. Kiro produced all seven specified test cases and ran the tests automatically, confirming all of them passed before I accepted and proceeded.

prompt 4:
“
Now organize everything into a Jupyter notebook with the following structure: a Markdown introduction cell explaining the module's purpose and design decisions, a cell with the full module code, a cell that writes both brasil_old.cnf and brasil_mercosul.cnf to disk programmatically, a cell that runs the full unittest suite and displays the results inline, and a

final Markdown cell summarizing the assumptions made. The notebook should be self contained, someone should be able to run it top to bottom and have everything work without any setup beyond having Python and Jupyter installed.
”

Explanation:

I chose to use a notebook to better showcase execution and results. I also asked Kiro to extensively document everything for easier code understanding and future adjustments if needed. I also chose to include a cell to write both configs into disk programmatically in the notebook since this ensures its truly self contained and eliminates environment dependent failure points. Here's also worth noting one adaption kiro made without being prompted, and that was to resolve the config file path via os.getcwd() rather than __file__, since jupyter notebooks lack that attribute. Kiro proposed a solution by resolving location through os.getcwd() inside the notebook, which returns directory from which the notebook is currently running. Changes made by Kiro were validated by running the notebook locally and confirming 7/7 tests passing in 0.007 seconds.

prompt 5:
“
Now create a README.md for this project containing: the purpose of the module and the design decisions behind putting validation constraints into config files rather than hardcoding them, an use example showing how to call validate_plate directly from Python, the structure and expected format of the .cnf config file, including which keys are optional and what happens when they are absent, how to run the standalone test suite, how to run the notebook, and a section documenting the assumptions made during implementation, specifically around character classification, case sensitivity, config file location resolution, and how unrecognized keys in the config are being handled.
”

Explanation:

With all artifacts in place I prompted Kiro to create readme documentation, serving as an entry point for anyone opening the repository and a good place for consolidating reasoning behind this implementation in one place. Output was reviewed to confirm accuracy and to ensure it reflected implementation.

Assumptions summary:

Among the assumptions made, was the one for character classification using str.isalpha() and str.isdigit(). These functions were used to count letters and digits respectively, since isalpha() returns true for both uppercase and lowercase letters, hence the validator was constructed under the assumption that it should be case insensitive, meaning that abc123 and ABC123 would satisfy the same constraints. Meanwhile, any character that doesn't classify as neither letter nor digit is silently ignored, only affecting the length constraint validation, if defined in the config. I made this assumption under the understanding that license plates, in practice, shouldn't have any differentiation over upper or lower case. Even though plates are often red only in upper case, having built in case insensitivity makes the module more defensive against possible edge cases. If a plate string comes arrives in lowercase, for any reason, the validation integrity is still kept. Normalization could also have been applied through a normalization step such as plate.upper() before validation, however, using isalpha() handles it implicitly without adding more code.

Another assumption made was that the solution should implement exactly only what the assignment specified, through what the config attributes allow for. Therefore, the decision not to add regex or positional validation was deliberate, since there are no requirements for it. In synthesis, the implementation is faithful to the spec.

The two last assumptions made concern the config handling. The first would be the config file location resolution, which was proposed by Kiro. In the standalone module, config files are resolved relative to the module file itself, using function that returns the directory where the module lives on disk. However, since I've also implemented Jupyter notebook to showcase execution, and Jupyter notebook files are not regarded as regular python files, it does not have file attribute, meaning same approach as in regular python files doesn't work. To accommodate for this, inside the notebook, config file location is resolved through os.getcwd(), which returns directory from which the notebook is currently running.

The last assumption is concerning unrecognized keys in the config file, the validator was designed as to read only recognized keys (length, letters, numbers) and silently ignore unrecognized ones, also making no attempt to validate the config structure itself. In practice, the config could contain additional fields such as metadata for another system and the validator would simply disregard and proceed normally. This translates to behavior such as if new keys are introduced in the future,

preexisting config files remain valid and the module continues operational without any code change, which I believe is the right approach for a module designed around an externally defined and potentially evolving config format.

B)

If in the future it was later required to make this module work standalone alongside implementing an interface, I would start by making the decision to implement a CLI wrapper around the existing module, preserving the current module itself, making no changes to its code. Given my previous implementation decisions, the module was developed in the sense that validate_plate takes a plate string, a config filename and returns a boolean. Taking this, we can take advantage of the fact that its already well suited to be called from any entry point without any requirement for code adjustments. Keeping the module itself and its code untouched is also beneficial since by doing this were also ensuring that existing integrations which import and call it directly continue to work as before, unaffected. This way, the CLI is merely an addition, a complement to what was already previously developed.

For developing the CLI wrapper itself, I would use the argparse library in python, for handling argument parsing and I would choose it because it requires no additional dependencies and is the standard approach for python CLI tools (https://docs.python.org/3/library/argparse.html). With the intent of mirroring the modules behavior, the interface would accept two positional arguments, the plate string and config file name. A basic invocation, using filenames and examples from the previous development would look something like:

python cli.py ABC1234 brasil_old.cnf

The expected output from such command would in turn be a message indicating if the plate is valid given the constraints or not, there would also be a non zero exit code, following the standard for cli tools behavior and convention across operating systems and shell, to treat cases of validation error such as in cases of missing config file or badly formatted JSON. In this cases, the error would be caught and surfaced as informative descriptive error messages, instead of only raw python tracebacks which are less useful. The exit code is also useful since it allows for the validator to be composed into shell scripts or automated pipelines, since any tool or script invoking the validator can programmatically check if it passed or failed, all this done without having to parse the output text. CLI exit codes by convention have 0

indicating success and any non-zero value indicating that something went wrong, this is useful for use cases such as a shell script that could chain the validator with other commands, only proceeding if the plate returns valid, or a CI pipeline could use it to validate a batch of plates and automatically fail the job if any returns a non-zero exit code.(https://www.redhat.com/en/blog/exit-codes-demystified)

Moreover, if this tool were to be distributed internally across teams, I would also package it using setuptools and a pyproject.toml, this way defining an entry point that users can invoke by name rather than calling the script directly with python, this follows the same approach I described in Q3 for the CLI deployment scenario. This would in turn make the process of installing and using the tool straightforward regardless of the users environment.

Q6)

The system here described is composed of a wrapper containing two inner modules, A and B. By looking at the execution runtime over time graph, we can conclude that the system ran stably at around 30 minutes from january through may, with only slight variations. However, the runtime started climbing sharply in June, reaching about 60 minutes, the double of previous runtimes by July. Looking at these metrics, and the pattern of a stable performance for a long period of time followed by sudden degradation, we can interpret it as a strong indicator that there was a specific change around June and July which led to the degradation, rather than just a gradual accumulation of technical debt. This is because gradual accumulation of debt would degrade runtime overtime but not so sharply and suddenly as in this scenario, this graph is much more indicative of something specific that led to the sharp degradation.

Having identified the unusual high rise in latency, the first step in investigating the cause for the degradation would be to establish a timeframe for when the degradation started, from the graph we can notice that the slope of the line increases sharply at around late may or early June, so that would be time window to be investigated. Given this window, that next step would be to look at what was changed during this period, looking up deployment history, code commits, dependencies updates, infrastructure changes, any configuration modifications. Git log or change management records within that time frame would be an useful resource in this investigation, since most cases of such behavior of sudden performance shift can be traced back to a specific change or commit.

Since the system consists of two inner modules, its also best to isolate which module is responsible for the degradation as well, since one of them may be solely responsible for the total runtime increase, eliminating the need to analyze the other. Even if its also possible that the degradation could be caused partially from both, diagnosing each module separately would also allow us to understand the contribution of each one to the total runtime increase, and focus investigation where it matters most. This can be done by recording the start and end time around specific blocks of code to measure how long those blocks take to execute. This would be code

instrumentation, its regarded as one of the simplest and more direct ways to identify where time is being spent in a system without relying on assumptions, however, since manual instrumentation itself introduces overhead, consuming additional cpu cycles to record measurements, and can clutter code, it should not be applied directly to prod, this would be done in staging environments replicating production. Placing those around each module to measure their individual executions times, would in turn make it clear whether the degradation is concentrated in one module or spread across both. Alternatives to manual instrumentation which avoid code modification include tools such as OpenTelemetry(auto instrumentation), an observability framework which tracks execution spans across code components without requiring manual timestamp recording, or eBPF(dynamic instrumentation), used to measure function level latency in production by hooking to function entry and exit points at the kernel level without modifying source code and with minimal overhead. (https://www.geeksforgeeks.org/system-design/latency-in-distributed-system/#measuring-latency , https://www.seas.upenn.edu/~ese5320/fall2020/handouts/hw2/profiling_tutorial.html#:~:text=make%20all%20./rendering-,4.2.,counter%20statistics%20of%20your%20program, https://signoz.io/opentelemetry/ , https://signoz.io/guides/what-is-api-latency/ , https://eunomia.dev/tutorials/33-funclatency/)

With the responsible module identified, we can proceed to narrow down the investigation by looking at the most likely causes of runtime increase. First thing to check, would of course be code change analysis through git history and commit logs. Having discarded code change, second thing would be whether the module is processing more data than before, if input volume grew significantly around June, than that in by itself could explain the degradation, especially if there are loops or databases queries that scale poorly with data size, such queries or loops by themselves could lead to such observed degradation in this scenario given data volume inflation. Third check would be external dependencies, since the impact that external dependencies have on overall system performance is well documented. Network latency, database query performance, and API response times are among the top contributors for system runtime degradation, so these are a good starting point for investigating. Moreover, external calls such as API requests or database queries should also be examined, as they may have been affected by factors such as network changes, increased load or a growing database returning results slower than before (https://zilliz.com/ai-faq/what-is-the-impact-of-latency-on-distributed-database-performance , https://sre.google/sre-book/monitoring-distributed-systems/ , https://www.geeksforgeeks.org/system-design/latency-in-distributed-system/ , https://pagertree.com/learn/devops/what-is-site-reliability-engineering-sre/four-golden-signals-sre-monitoring).

Proceeding with our analysis, if the code itself appears unchanged, we should proceed to look at the infrastructure level. Even without code modifications, servers runtime running low on memory, disks nearing capacity, or competing processes consuming CPU can also silently degrade performance over time. For this reason, its

useful to check resource usage metrics during execution, to discard these factors as possibilities for the degradation.

For a complementary analysis, tools such as code profiling can complement previous instrumentation methods by taking a distinct approach, rather than measuring specific metrics as in instrumentation, a profiler is able to observe the entire program automatically as it runs, collecting execution statistics across all functions simultaneously. This is particularly useful when system bottleneck is still unknown, for it provides us with a complete picture of where time is being spent across the entire codebase. (https://www.geeksforgeeks.org/system-design/tracing-and-profiling-techniques-for-distributed-systems/)

After all analysis, once the root cause for degradation is identified, its also important to perform the fix first in a staging environment and validate it properly before being applied to prod. And thinking long term, automated runtime monitoring with alerting thresholds and modular execution logging would also ensure that similar degradations are caught much earlier in the future, minimizing their impacts. (https://sre.google/sre-book/monitoring-distributed-systems/)